

파이썬 기초 프로그래밍과 응용

250218 - 2일차

경로 최적화란 무엇인가?

출발지와 목적지 간의 이동 경로를 가장 효율적으로 설계하는 과정

- 이동 시간 단축
- 자원 활용도 극대화

경로 최적화란 무엇인가?

- 물류 및 배송: 여러 배송 지점을 효율적으로 방문하기 위한 경로 설계.
- 네트워크 통신: 데이터 패킷이 가장 빠르고 안정적으로 전달될 수 있는 경로 탐색.
- 대중교통 및 자율주행: 실시간 교통 상황에 따라 최적의 이동 경로 제공.

경로 최적화를 위한 알고리즘

- BFS(Breadth-First search)
- DFS(Depth-First Search)
- 다익스트라 (Dijkstra) 알고리즘
- A* 알고리즘

그래프 (Graph)

정점(Vertex, Node)과 이를 연결하는 간선(Edge)으로 구성된 수학적 구조.



```
graph_matrix = [  
    [0, 1, 1, 0, 0, 0],  
    [0, 0, 0, 1, 1, 0],  
    [0, 0, 0, 0, 0, 1],  
    [0, 0, 0, 0, 0, 0],  
    [0, 0, 0, 0, 0, 0],  
    [0, 0, 0, 0, 0, 0]  
]
```

```
graph_matrix = [  
    [0, 1, 1, 0, 0, 0],  
    [1, 0, 0, 1, 1, 0],  
    [1, 0, 0, 0, 0, 1],  
    [0, 1, 0, 0, 0, 0],  
    [0, 1, 0, 0, 0, 0],  
    [0, 0, 1, 0, 0, 0]  
]
```

그래프 (Graph)

정점(Vertex, Node)과 이를 연결하는 간선(Edge)으로 구성된 수학적 구조.



```
graph_dict = {  
    1: [2, 3],  
    2: [4, 5],  
    3: [6]  
}
```

```
graph_dict = {  
    1: [2, 3],  
    2: [1, 4, 5],  
    3: [1, 6],  
    4: [2],  
    5: [2],  
    6: [3]  
}
```

너비 우선 탐색 (BFS, Breadth-First search)

- 가까운 노드부터 차례로 탐색하는 방식.
- 주로 최단 경로 탐색이나 연결 여부 확인에 사용.
- Queue(큐) 자료구조를 사용.
 - 선입선출(FIFO: First In, First Out) 원칙을 따르는 선형 자료구조
 - 먼저 들어온 데이터가 먼저 처리



너비 우선 탐색 (BFS, Breadth-First search)

1. 시작 노드를 큐에 삽입.
2. 큐에서 노드를 꺼내고 방문 처리.
3. 해당 노드의 인접 노드 중 방문하지 않은 노드를 큐에 삽입.
4. 큐가 빌 때까지 반복.

너비 우선 탐색 (BFS, Breadth-First search)



• (.venv) PS D:\marker-main> python .\5_bfs_cli.py

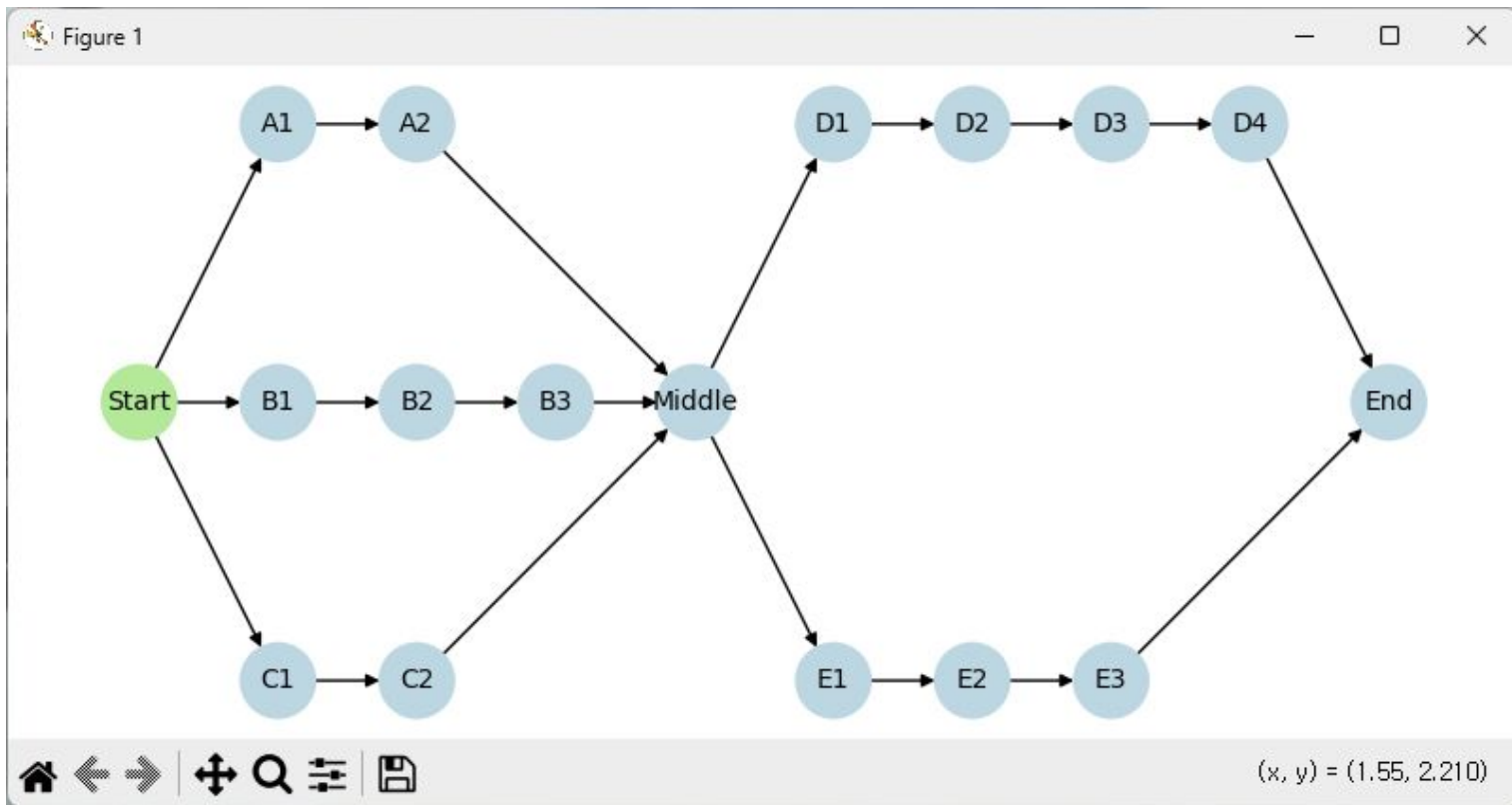
BFS는 다음과 같은 순서로 탐색을 합니다.

- 1 노드를 방문했습니다.
- 2 노드를 방문했습니다.
- 3 노드를 방문했습니다.
- 4 노드를 방문했습니다.
- 5 노드를 방문했습니다.
- 6 노드를 방문했습니다.

BFS는 다음과 같은 순서로 탐색을 합니다.

- 5 노드를 방문했습니다.
- 2 노드를 방문했습니다.
- 1 노드를 방문했습니다.
- 4 노드를 방문했습니다.
- 3 노드를 방문했습니다.
- 6 노드를 방문했습니다.

너비 우선 탐색 (BFS, Breadth-First search)

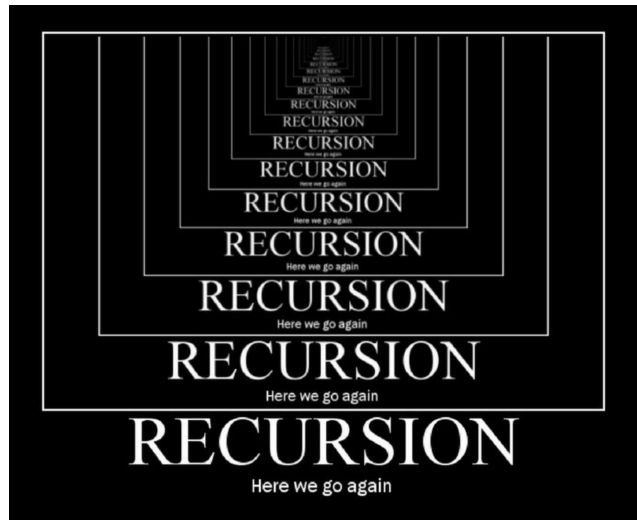


너비 우선 탐색 (BFS, Breadth-First search)

- 장점: 가중치 없는 그래프에서 최단 경로를 보장.
- 단점: 메모리 소비가 크며, 큰 그래프에서는 비효율적.

깊이 우선 탐색 (DFS, Depth-First Search)

- 최대한 깊이 들어간 후 되돌아오며(backtracking) 탐색하는 방식.
- 스택(Stack) 자료구조를 사용하여 구현
 - 또는 재귀 호출(Recursive)로 구현 가능



깊이 우선 탐색 (DFS, Depth-First Search)

1. 시작 노드를 방문하고 스택에 추가.
2. 방문한 노드의 인접 노드 중 방문하지 않은 노드를 스택에 추가.
3. 더 이상 방문할 노드가 없으면 이전 노드로 돌아감.
4. 모든 노드를 방문할 때까지 반복.

깊이 우선 탐색 (DFS, Depth-First Search)



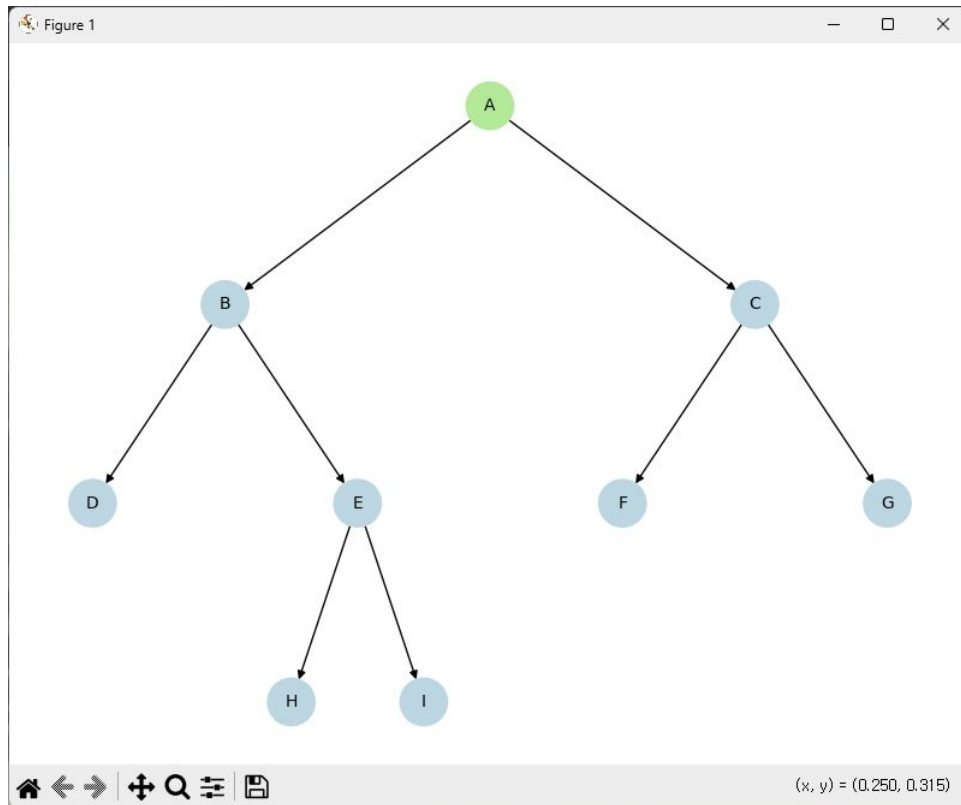
• (.venv) PS D:\marker-main> python .\7_dfs_cli.py
DFS는 다음과 같은 순서로 탐색을 합니다.

- 1 노드를 방문했습니다.
- 2 노드를 방문했습니다.
- 4 노드를 방문했습니다.
- 5 노드를 방문했습니다.
- 3 노드를 방문했습니다.
- 6 노드를 방문했습니다.

DFS는 다음과 같은 순서로 탐색을 합니다.

- 5 노드를 방문했습니다.
- 2 노드를 방문했습니다.
- 1 노드를 방문했습니다.
- 3 노드를 방문했습니다.
- 6 노드를 방문했습니다.
- 4 노드를 방문했습니다.

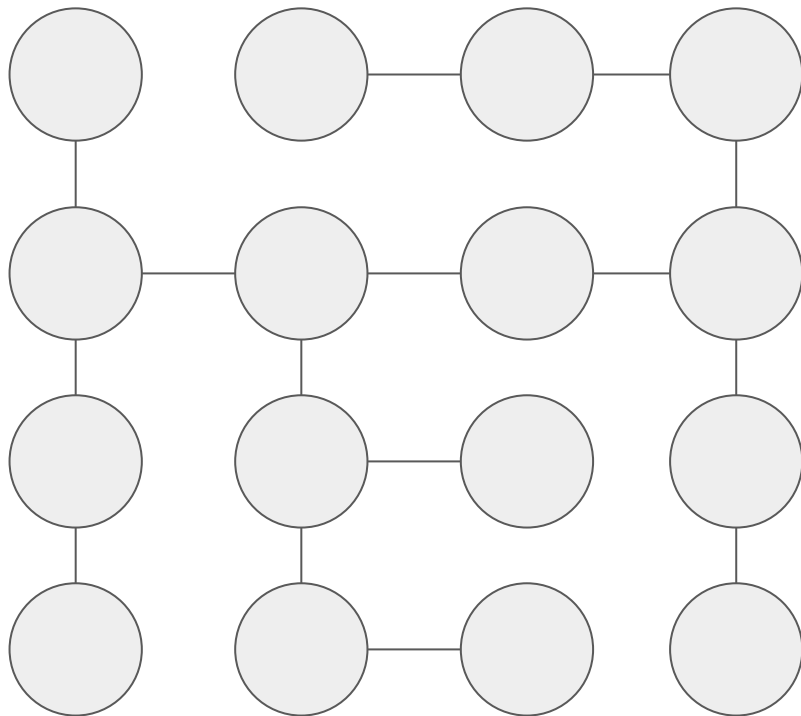
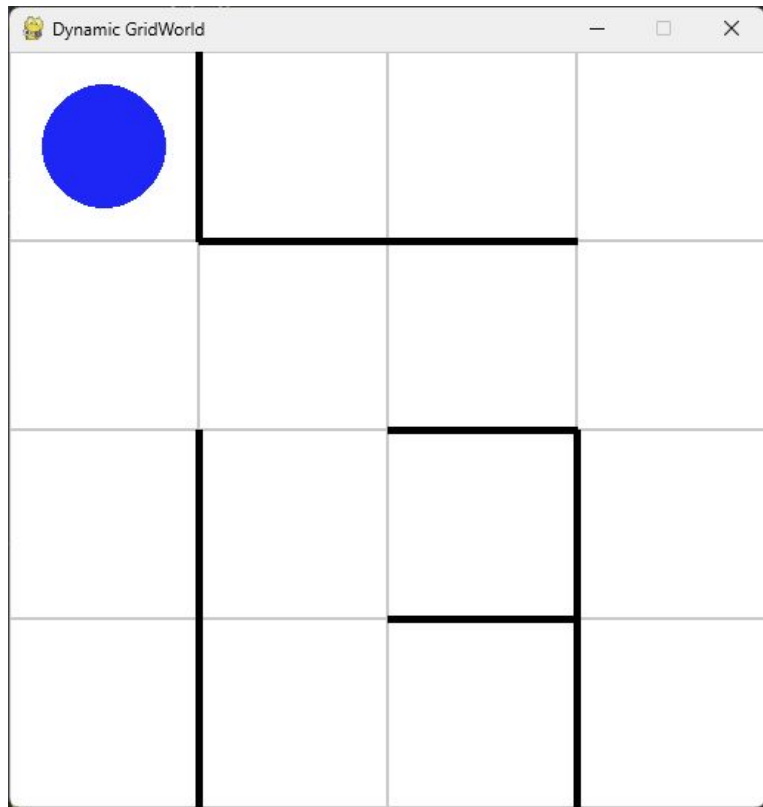
깊이 우선 탐색 (DFS, Depth-First Search)



깊이 우선 탐색 (DFS, Depth-First Search)

- 장점: 간단한 구현, 메모리 효율적(인접 리스트 사용 시).
- 단점: 최단 경로를 보장하지 않으며, 사이클에 의한 무한 루프 방지를 위해 방문 기록 필요.

BFS, DFS 만들어보기



함수형 프로그래밍 (Functional Programming)

$$\begin{aligned}
 |a| &= \begin{cases} a, a \geq 0 \\ -a, a < 0 \end{cases} & u_i &= R_i i_i + \sum_{j=1}^{2q} L_{i,j} \frac{di_j}{dt} + \omega \sum_{j=1}^{2q} i_j \frac{dL_{i,j}}{d\phi} \\
 (a-b)(a^2+ab+b^2) &= a^3 - b^3 & \int x^\alpha \cdot dx &= \frac{x^{\alpha+1}}{\alpha+1} + c & \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \\
 \sin \frac{\alpha}{2} \cdot \cos \frac{\alpha}{2} &\cdot \sqrt{\sum_{i=1}^n (x_i - y_i)^2} & (x^n)' &= nx^{n-1} & \frac{\sqrt[n]{a}}{\sqrt[n]{b}} &= \sqrt[n]{\frac{a}{b}} & \sin \alpha &= 2 \sin \frac{\alpha}{2} \cos \frac{\alpha}{2} \\
 i_i \sum_{j=1}^{2q} i_j \frac{dL_{i,j}}{d\phi} &\cdot \sqrt{\sum_{i=1}^n (x_i - y_i)^2} & \frac{\pi}{2} - \text{ArcSin}(x) & & u_i &= R_i i_i + \sum_{j=1}^{2q} L_{i,j} \frac{di_j}{dt} + \omega \sum_{j=1}^{2q} i_j \frac{dL_{i,j}}{d\phi} \\
 y = x \times 2 & & & & & & & \\
 \sin \alpha = 2 \sin \frac{\alpha}{2} \cdot \cos \frac{\alpha}{2} &\cdot \text{ctg} \alpha + \text{ctg} \beta = \frac{\sin(\alpha + \beta)}{\cos(\alpha + \beta)} & x_{1,2} &= \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}
 \end{aligned}$$

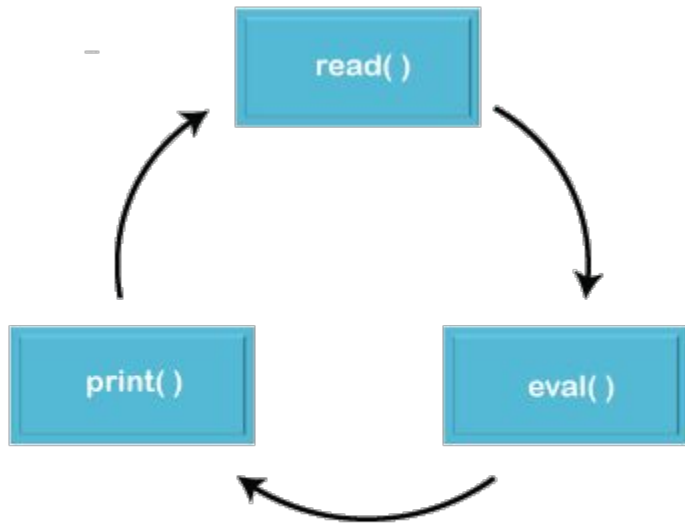
VectorStock

VectorStock.com/23990015

함수형 프로그래밍 (Functional Programming)

- 수학적 함수에 기반한 프로그래밍 패러다임
- 코드가 간결하고, 테스트와 디버깅이 용이
- 병렬 처리가 자연스럽고, 상태 관리가 용이
- 부작용이 적어 예측 가능한 코드를 유지

함수형 프로그래밍 (Functional Programming)



Read-Eval-Print Loop (REPL)

함수형 프로그래밍 (Functional Programming)

- Read-Eval-Print Loop의 약자
- 코드를 입력하면 즉시 실행 결과를 반환하는 대화형 환경
 - Read: 코드를 읽음
 - Eval: 코드를 평가(실행)
 - Print: 결과를 출력
 - Loop: 반복

함수형 프로그래밍 특징

- 순수 함수
 - 같은 입력에 대해 항상 같은 출력을 반환하며, 부작용이 없는 함수를 지향.
- 불변성
 - 데이터는 변경되지 않으며, 새로운 데이터를 생성하여 상태를 유지
- 고차 함수
 - 함수를 인자로 받거나, 함수를 반환하는 함수
 - 함수의 조합 가능성과 재사용성이 상승
- 지연 평가
 - 필요할 때까지 계산을 미루어 성능을 최적화

리스트 다루기 - 이어 붙이기

```
1  # 리스트 이어붙이기
2  list1 = [1, 2, 3]
3  list2 = [4, 5, 6]
4  concatenated_list = list1 + list2
5  print(concatenated_list) # [1, 2, 3, 4, 5, 6]
```

리스트 다루기 - 일괄 작업

```
1  # 리스트의 모든 요소에 특정 작업을 하기
2  # for 반복문을 통해서 하는 방법
3  numbers = [1, 2, 3, 4, 5]
4  increased_numbers = []
5  for number in numbers:
6      |   increased_numbers.append(number**2 + 2)
7  print(increased_numbers)  # [3, 6, 11, 18, 27]
```


리스트 다루기 - 일괄 작업

```
1  # 리스트의 모든 요소에 특정 작업을 하기
2  # 리스트 컴프리헨션 방법
3  numbers = [1, 2, 3, 4, 5]
4  increased_numbers = [number**2 + 2 for number in numbers]
5  print(increased_numbers) # [3, 6, 11, 18, 27]
```

리스트 컴프리헨션 (Comprehension)

```
1  numbers = [1, 2, 3, 4, 5]
2  increased_numbers = []
3  for number in numbers:
4      |    increased_numbers.append(number**2 + 2)
5
6  increased_numbers = [number**2 + 2 for number in numbers]
```

익명함수, 람다 (lambda)

```
lambda 인자: 인자 * 2 # 작업
```

익명함수, 람다 (lambda)

```
1  add = lambda x, y: x + y
2  result = add(2, 3)
3  print(result)  # 5
```

리스트 다루기 - 일괄 작업

```
1  # 리스트의 모든 요소에 특정 작업을 하기
2  # map 과 람다(lambda) 를 이용한 방법
3  numbers = [1, 2, 3, 4, 5]
4  squared_numbers = list(map(lambda x: x**2 + 2, numbers))
5  print(squared_numbers)  # [3, 6, 11, 18, 27]
```

리스트 다루기 - 정렬

```
1  # 리스트 정렬하기
2  unsorted_list = [5, 2, 9, 1, 5, 6]
3  sorted_list = sorted(unsorted_list)
4  print(sorted_list) # [1, 2, 5, 5, 6, 9]
```

리스트 다루기 - 조건으로 남기기

```
1  # 리스트에서 특정 조건의 요소를 남기기
2  # 리스트 컴프리헨션 방법
3  numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
4  even_numbers = [x for x in numbers if x % 2 == 0]
5  print(even_numbers)  # [2, 4, 6, 8]
```

리스트 다루기 - 조건으로 남기기

```
1  # 리스트에서 특정 조건의 요소를 남기기
2  # for 루프를 쓰는 방법
3  numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
4  even_numbers = []
5  for x in numbers:
6      if x % 2 == 0:
7          even_numbers.append(x)
8  print(even_numbers) # [2, 4, 6, 8]
```


리스트 다루기 - 조건으로 남기기

```
1  # 리스트에서 특정 조건의 요소를 남기기
2  # filter 과 람다(lambda) 를 이용한 방법
3  numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
4  even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
5  print(even_numbers)  # [2, 4, 6, 8]
```

리스트 다루기 - 평탄화 (flatten)

```
1  # 중첩된 2차원 리스트를 1차원 리스트로 만들기
2  nested_list = [[1, 2, 3], [4, 5], [6, 7, 8]]
3  flattened_list = sum(nested_list, [])
4  print(flattened_list)  # [1, 2, 3, 4, 5, 6, 7, 8]
```

경로 탐색 알고리즘의 함수화

```
• (.venv) PS D:\marker-main> python .\9_bfs_functional.py
1 노드를 방문했습니다.
2 노드를 방문했습니다.
3 노드를 방문했습니다.
4 노드를 방문했습니다.
5 노드를 방문했습니다.
6 노드를 방문했습니다.
BFS 방문 순서: [0, 1, 2, 3, 4, 5]
```

```
• (.venv) PS D:\marker-main> python .\10_dfs_functional.py
1 노드를 방문했습니다.
2 노드를 방문했습니다.
4 노드를 방문했습니다.
5 노드를 방문했습니다.
3 노드를 방문했습니다.
6 노드를 방문했습니다.
DFS 방문 순서: [0, 1, 3, 4, 2, 5]
```

감사합니다